THE UNIVERSITY OF MELBOURNE

SWEN30006

Software Modelling and Design

Applying GoF Design Patterns

Part 2: Composite pattern

Revisit strategy, template, observer patterns

Anything you can do, I can do meta.

—Daniel Dennett

Lecturer: Patanamon Thongtanunam

Book ref.: Larman Chapter 26

Objectives

On completion of this topic you should be able to:

Apply some GoF design patterns

- Façade : Covered in Week 7 Lecture
- Decorator : Covered in Week 7 Lecture
- Builder : Covered in Week 7 Lecture
- Adapter : Covered in Week 8 Lecture
- Singleton : Covered in Week 8 Lecture
- (Concrete) Factory : Covered in Week 8 Lecture
- **Composite : Discussed in this Week's Lecture (Week 9)**
- **Template, Strategy, Observer : Revisited from SWEN20003 (Week 9)**

Recognise GRASP principles as a generalization of other design patterns.

We don't cover all GoF 23 design patterns in this course.

Problem 2.1: Complex Pricing Logic

- POS provides more complex pricing logic, e.g.,
  - store-wide discount for the day
  - senior citizen discounts
- The pricing strategy for a sale can vary, e.g.,
  - one period it may be 10% off all sales
  - later it may be $10 off if the sale total is greater than $200

Strategy (GoF) – Revisited from SWEN20003

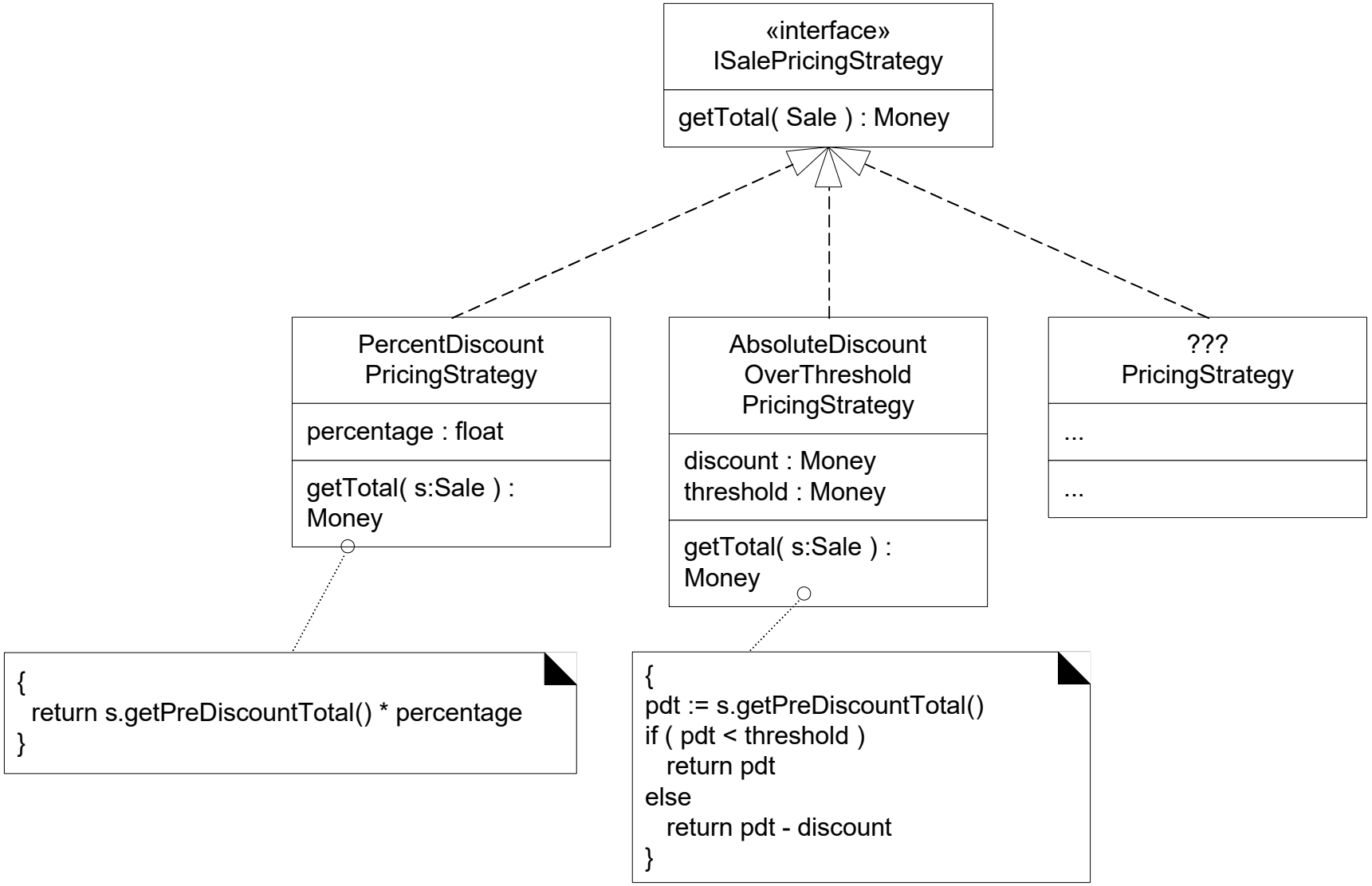
Problem:

- How to design for varying, but related, algorithms or policies?
- How to design for the ability to change these algorithms or policies?

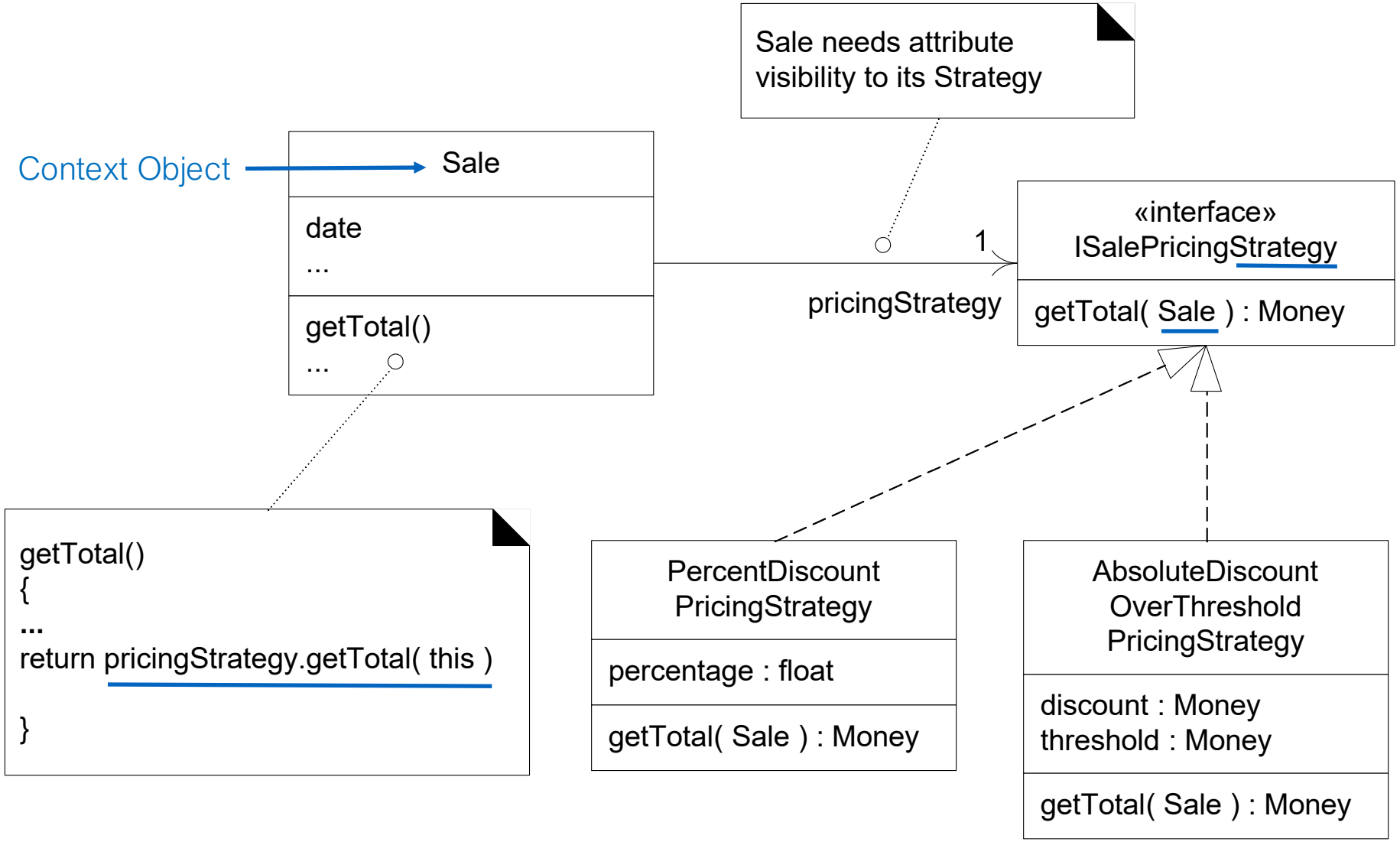
Solution (advice):

- Define each algorithm/policy/strategy in a separate class, with a common interface.

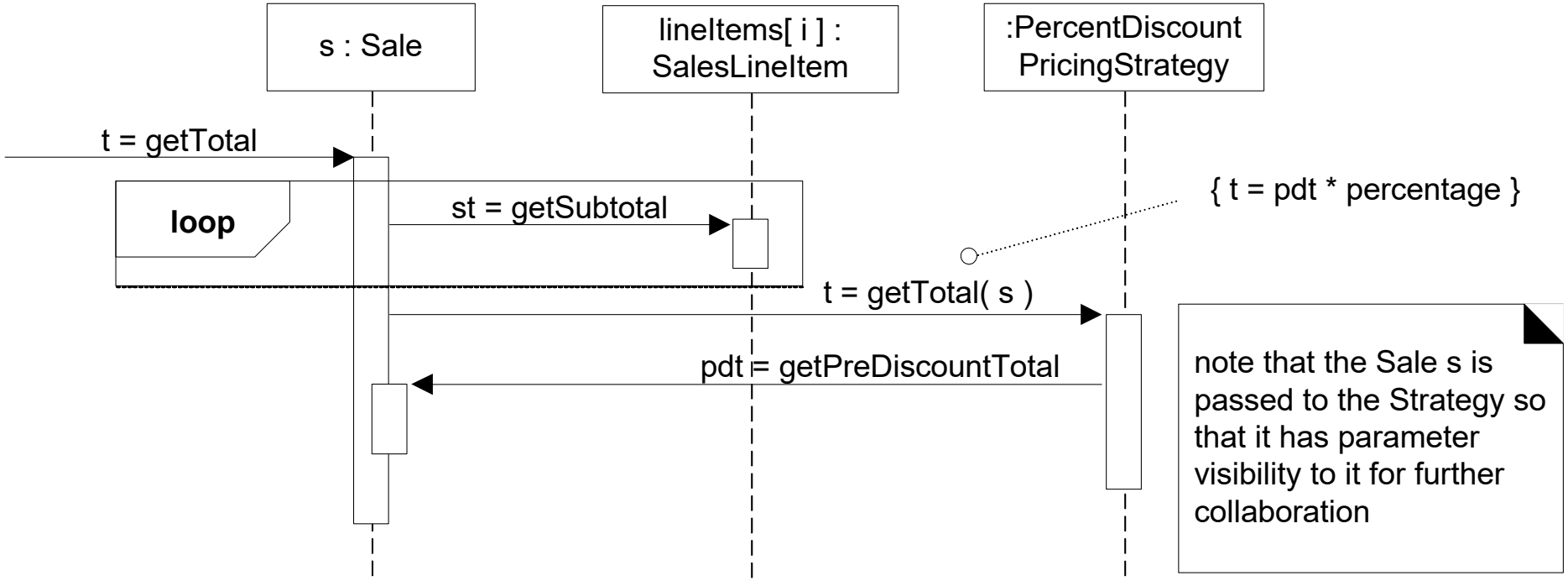
Pricing Strategy Classes



Context Object Visibility to Strategy



Strategy in Collaboration



Example

```
public interface ISalePricingStrategy {
        public double getTotal(Sale s);
}
```

ISalePricingStrategy.java

```
public class PercentDiscountPricingStrategy implements ISalePricingStrategy{
        private double percentage = 0.05;
        public double getTotal(Sale s) {
                return s.getPreDiscountTotal() - (s.getPreDiscountTotal() * percentage);
        }
}
```

PercentDiscountPricingStrategy.java

Example (cont)

```
public class AbsoluteDiscountOverThresholdPricingStrategy implements ISalePricingStrategy {
        private double threshold = 50;
        private double discount = 5;
        public double getTotal(Sale s) {
                double pdt = s.getPreDiscountTotal();
                if (pdt >= threshold) {
                        return pdt - discount;
                } else {
                        return pdt;
                }
        }
}
```

AbsoluteDiscountOverTheThresholdPricingStrategy.java

Which of the following statement is **FALSE**?

0

- Both Strategy and Adapter patterns offer Protected Variation since various algori…
- Strategy and Adapter can be referred interchangeably since both patterns are bas…
- Adapter uses Indirection, while Strategy doesn't use Indirection.
- Adapter is a structural design pattern and Strategy is a behavioural design pattern.

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Which of the following statement is **FALSE**?

0

Both Strategy and Adapter patterns offer Protected Variation since various algorithms or interfaces can be used through a single interface object.

0%

Strategy and Adapter can be referred interchangeably since both patterns are based on Polymorphism .

0%

Adapter uses Indirection, while Strategy doesn't use Indirection.

0%

Adapter is a structural design pattern and Strategy is a behavioural design pattern.

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Which of the following statement is **FALSE**?

0

Both Strategy and Adapter patterns offer Protected Variation since various algorithms or interfaces can be used through a single interface object.

0%

Strategy and Adapter can be referred interchangeably since both patterns are based on Polymorphism .

0%

Adapter uses Indirection, while Strategy doesn't use Indirection.

0%

Adapter is a structural design pattern and Strategy is a behavioural design pattern.

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Problem 2.2: Multiple Conflicting Policies

E.g. suppose the stores pricing today (Monday) is:

- 20% senior discount policy
- preferred customer discount: 15% off sales over $400
- on Monday, there is $50 off purchases over $500
- buy 1 case Darjeeling tea, get 15% discount off everything

Factors:

1. customer type (senior, preferred)
2. time period (Monday)
3. line item product (Darjeeling tea)

Combining Policies

conflict resolution strategy:

- when multiple policies are applicable, how are these policies resolved?
  - Some discounts **cannot be combined** with others
  - Possible policies: Best for customer or Best for store

Composite pricing strategy:

- Determine which pricing strategies are applicable
- Apply the relevant conflict resolution strategy

Composite (GoF)

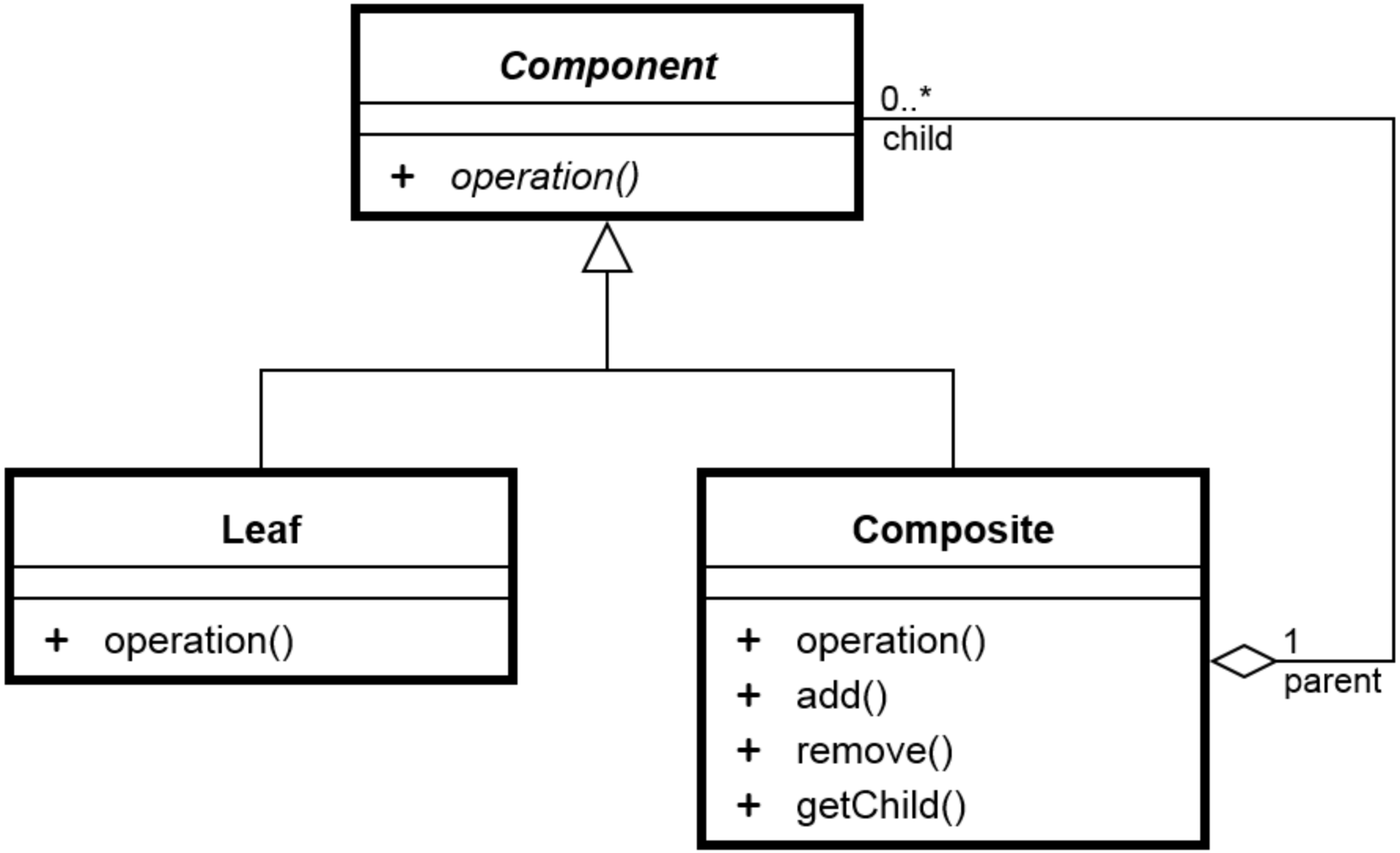
Problem:

- How to treat a group or composition structure of objects the same way (polymorphically) as a non-composite (atomic) object?

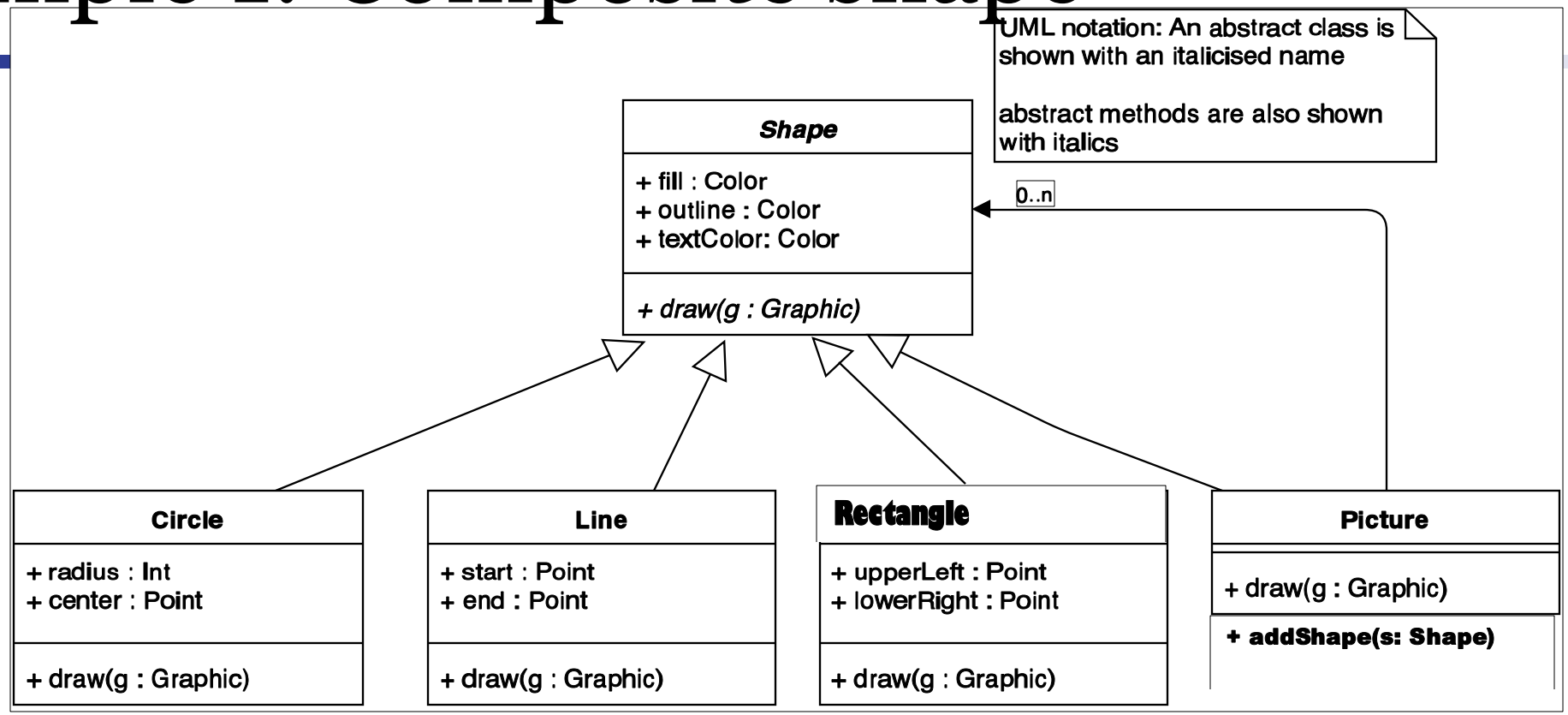
Solution (advice):

- Define classes for composite and atomic objects so that they implement the same interface.

Composite: Generalised Structure



Example 1: Composite shape



`Circle.draw(g)` `Line.draw(g)` `Rectangle.draw(g)` `Picture.draw(g)`

Example: Main

```
public class CompositeDrawing {
  public static void main(String[] args) {
  Circle circle1 = new Circle(5,new Point(0,0));
  Rectangle rectangle1 = new Rectangle(new Point(0,10),new Point(10,20));
  Line line1 = new Line(new Point(5,10),new Point(0,10));
  Picture myPicture = new Picture();

  myPicture.addShape(circle1);
  myPicture.addShape(rectangle1);
  myPicture.addShape(line1);
  myPicture.draw();
 }
}
```

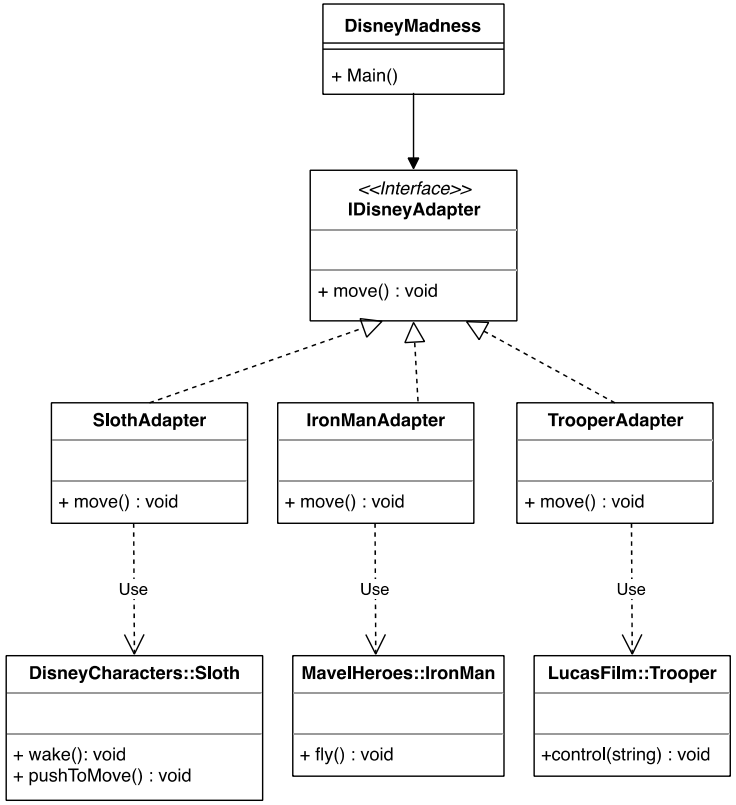
Example: Picture (Composite Shapes)

```
public class Picture extends Shape{
  ArrayList<Shape> shapes;
  public Picture() {
    shapes = new ArrayList<Shape>();
  }

  public void addShape(Shape shape) {
   shapes.add(shape);
  }

  public void draw() {
   for(Shape s: shapes) {
     s.draw();
   }
  }
}
```

Example2: Composite characters

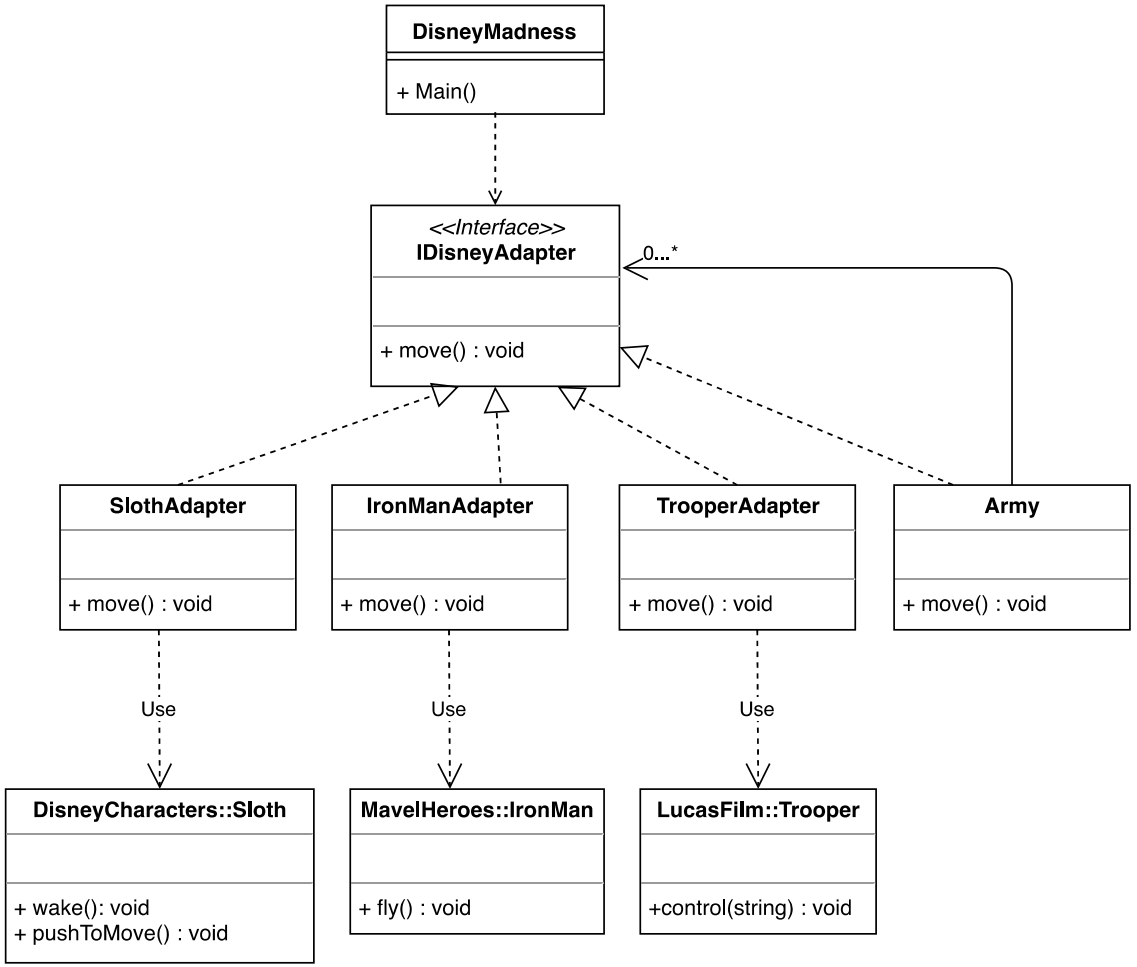
- Create a group of characters for DisneyMadness
  - Update the current diagram to use composite

Disney Army



How to extend the current design?

Example 2: Composite characters



(Problem 2.2) Combining Policies

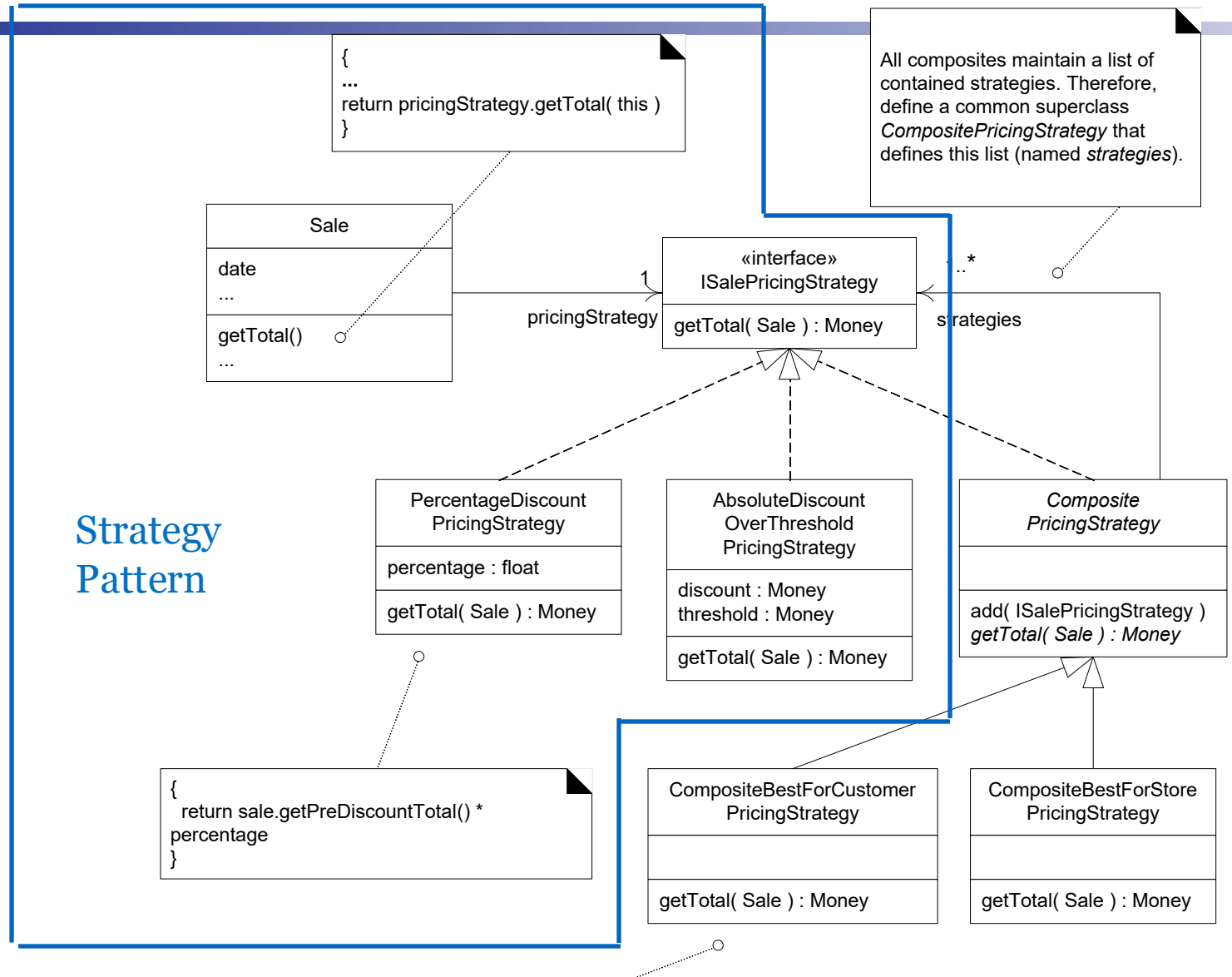
Conflict resolution strategy:

- when multiple policies are applicable, how are these policies resolved?
  - Some discounts cannot be combined with others
  - **Possible policies: Best for customer or Best for store**

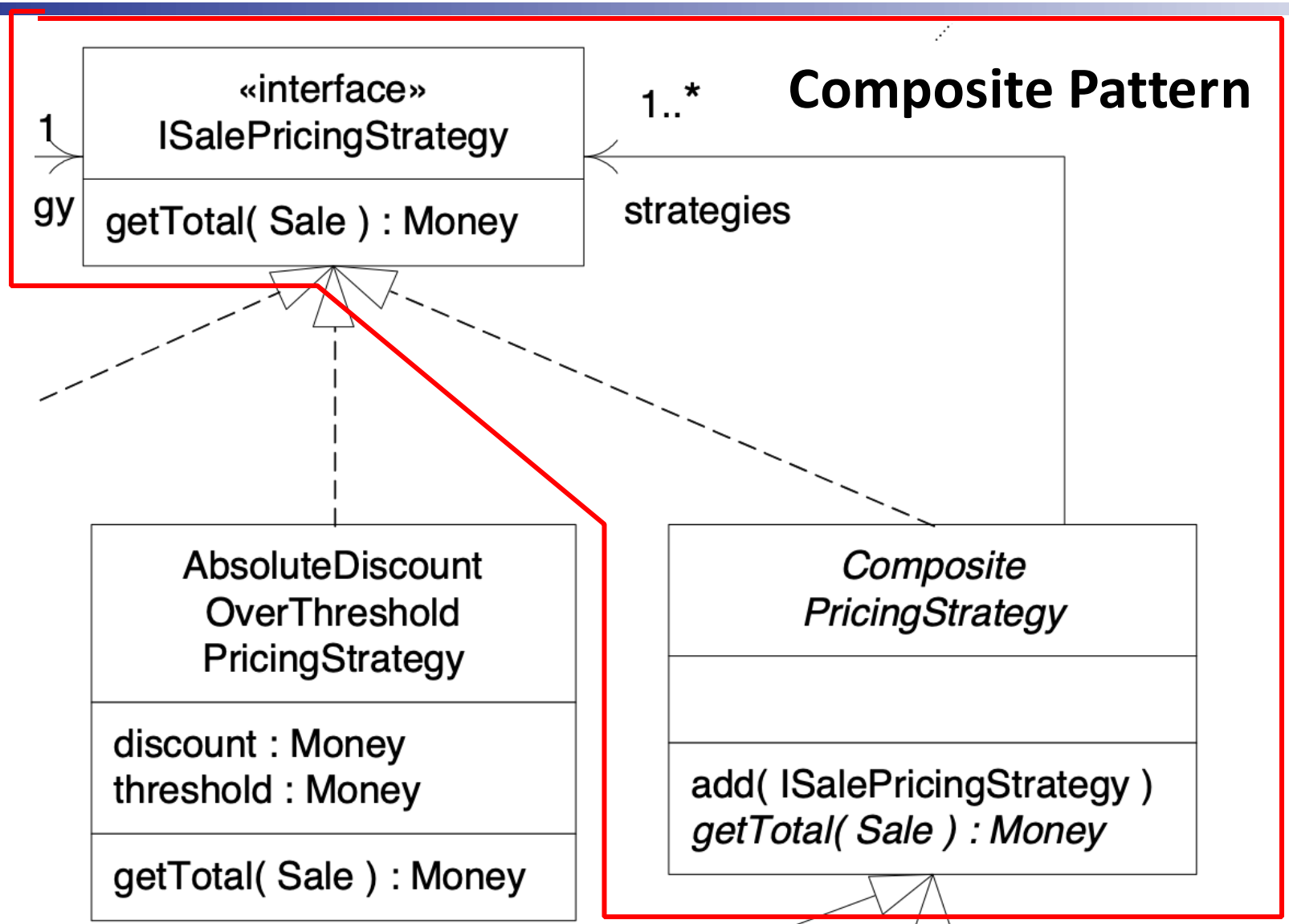
Composite pricing strategy:

- Determine which pricing strategies are applicable
- Apply the relevant conflict resolution strategy

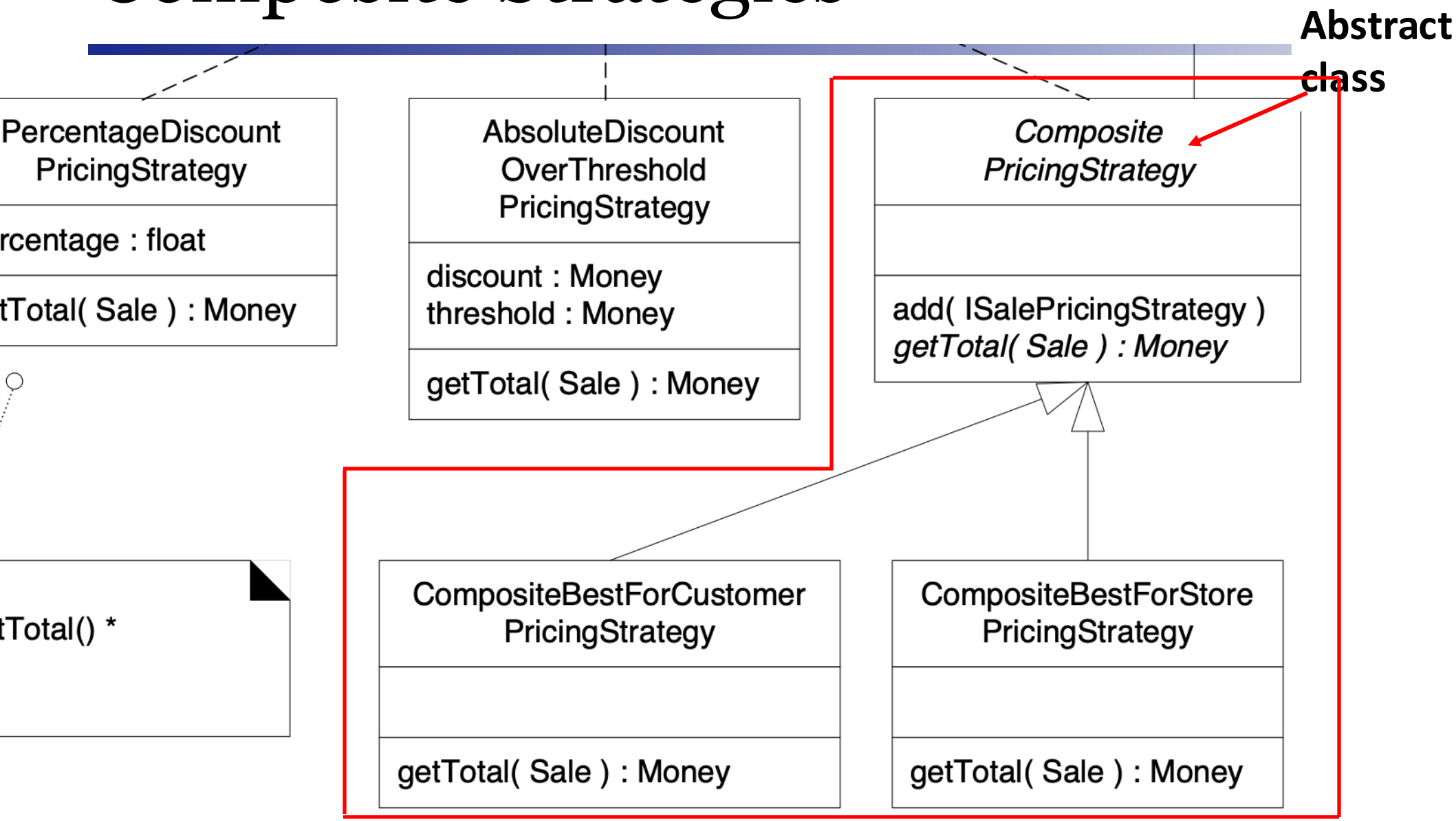
Composite Strategies



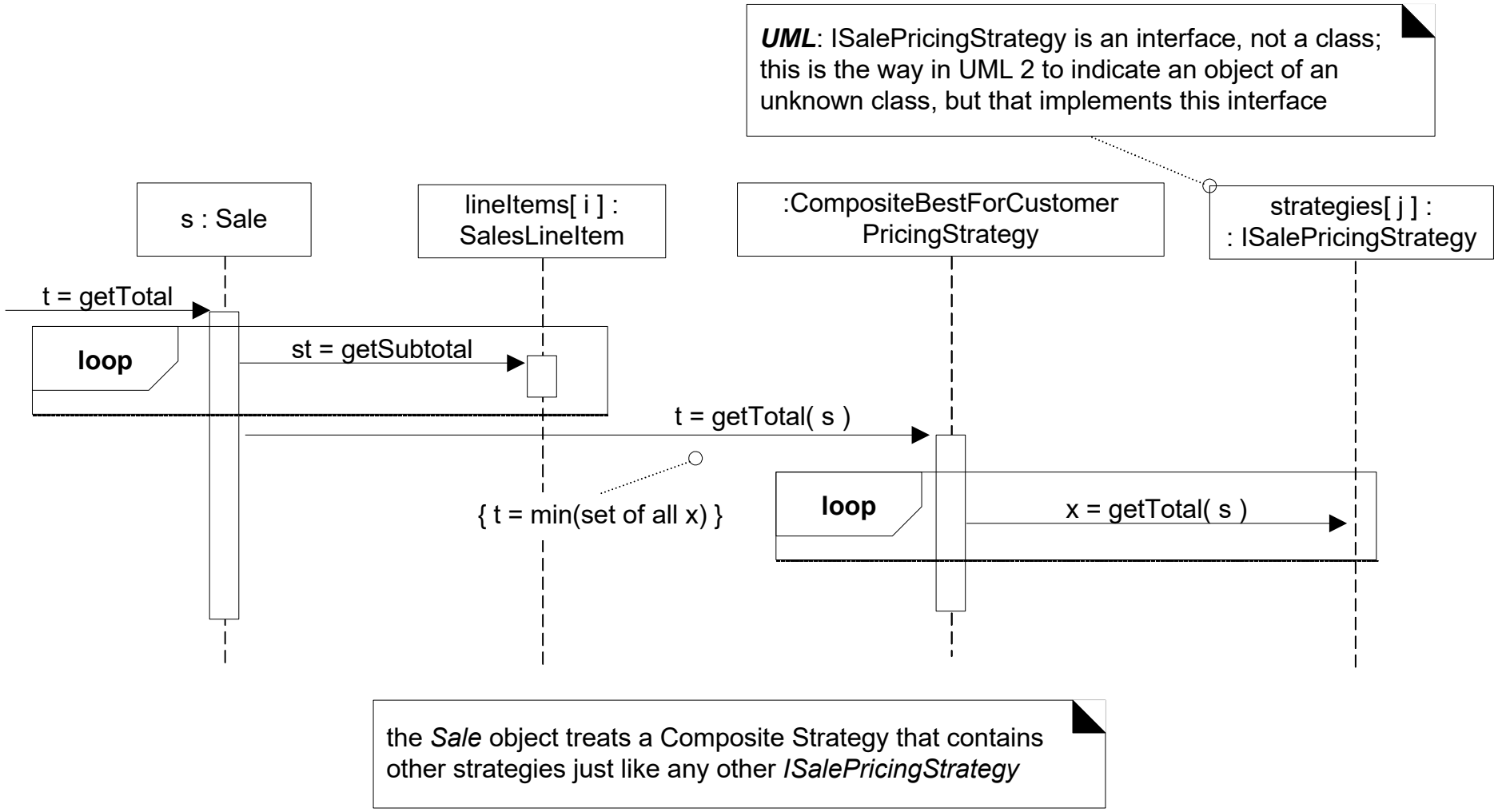
Composite Strategies



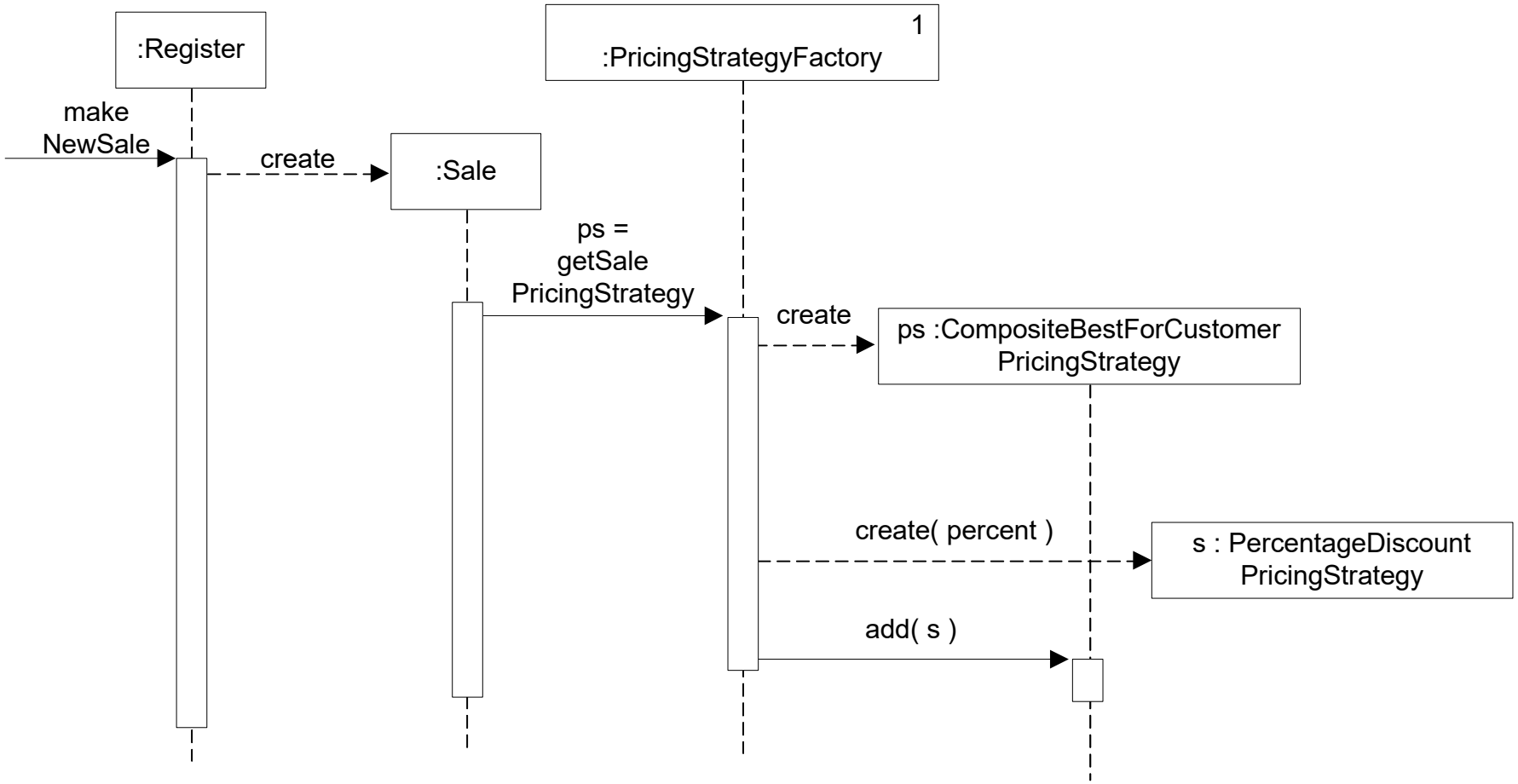
Composite Strategies



Collaboration with a Composite



Creating a Composite Strategy



Example

```
public interface ISalePricingStrategy {
        public double getTotal(Sale s);
        public String getStrategyName();
}
```

ISalePricingStrategy.java

```
abstract class CompositePricingStrategy implements ISalePricingStrategy {

        protected ArrayList<ISalePricingStrategy> pricingStrategies =
                new ArrayList<ISalePricingStrategy>();

        public void add(ISalePricingStrategy strategy) {
            pricingStrategies.add(strategy);
        }

        public abstract double getTotal(Sale s);
}
```

CompositePricingStrategy.java

Example: Composite Strategy

```
public class CompositeBestForCustomerPricingStrategy extends CompositePricingStrategy {
  private String selectedStrategy = null;
  //Get minimum total
  public double getTotal(Sale s) {
    double lowestTotal = s.getPreDiscountTotal();

    for(ISalePricingStrategy strat: this.pricingStrategies){
            double total = strat.getTotal(s);
            if (lowestTotal > total) {
                    lowestTotal = total;
            }
    }
    return lowestTotal;
  }
}
```

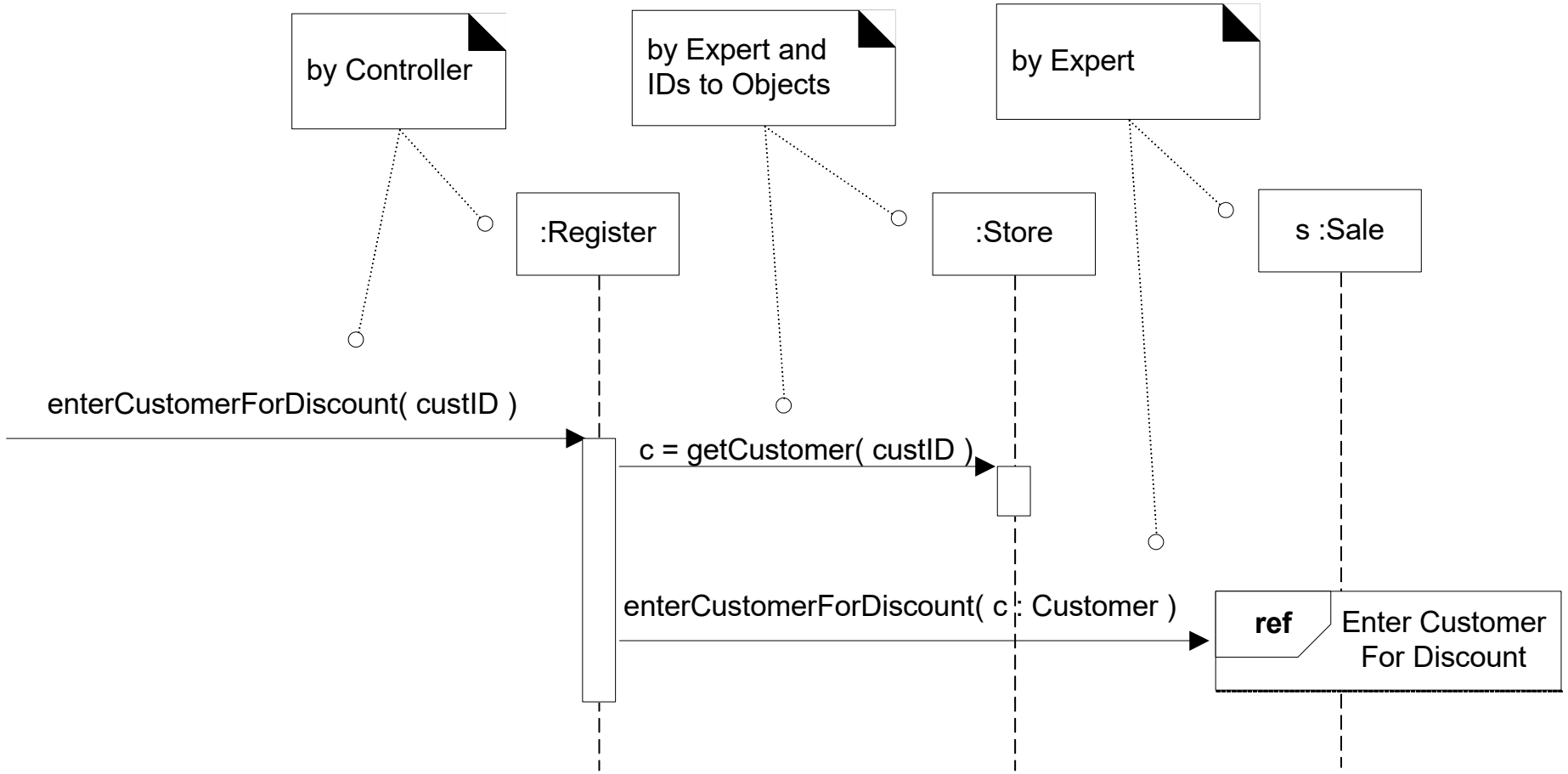
Example: Composite Strategy Factory

```
public class PricingStrategyFactory {
  public ISalePricingStrategy getCompositeSalePricingStrategy() {
   ISalePricingStrategy applicableStrategies = null;
   LocalDateTime today = LocalDateTime.now();
   switch(today.getDayOfWeek()) {
     case MONDAY:
        CompositePricingStrategy composStrat = new CompositeBestForCustomerPricingStrategy();
        composStrat.add(new PercentDiscountPricingStrategy());
        composStrat.add(new TeaDiscountPricingStrategy());
        applicableStrategies = composStrat;
        break;
      case FRIDAY:
         applicableStrategies = new AbsoluteDiscountOverThresholdPricingStrategy();
         break;
}
return applicableStrategies;
  }
}
```

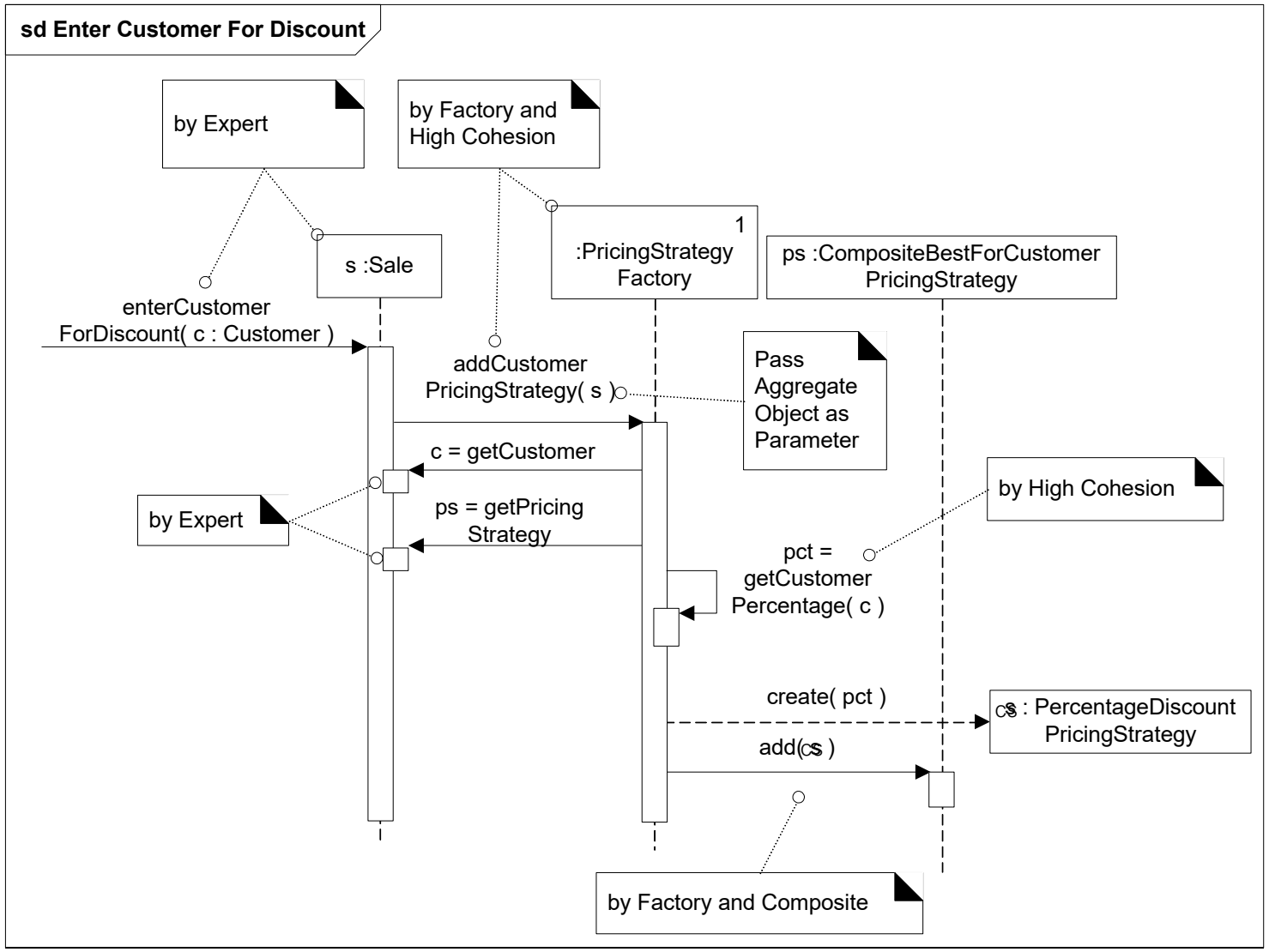
Extension (or Alternative Flows)

- Customer says they are eligible for a discount (e.g., employee, preferred customer)
  - Cashier signals discount request.
  - Cashier enters Customer identification.
  - System presents discount total, based on discount rules.

Creating Discount Pricing Strategy (1)



Creating Discount Pricing Strategy (2)



Summary: Complex Pricing Logic

- POS has various pricing strategies/discounts and we want to be able to combine them flexibly

Solution:

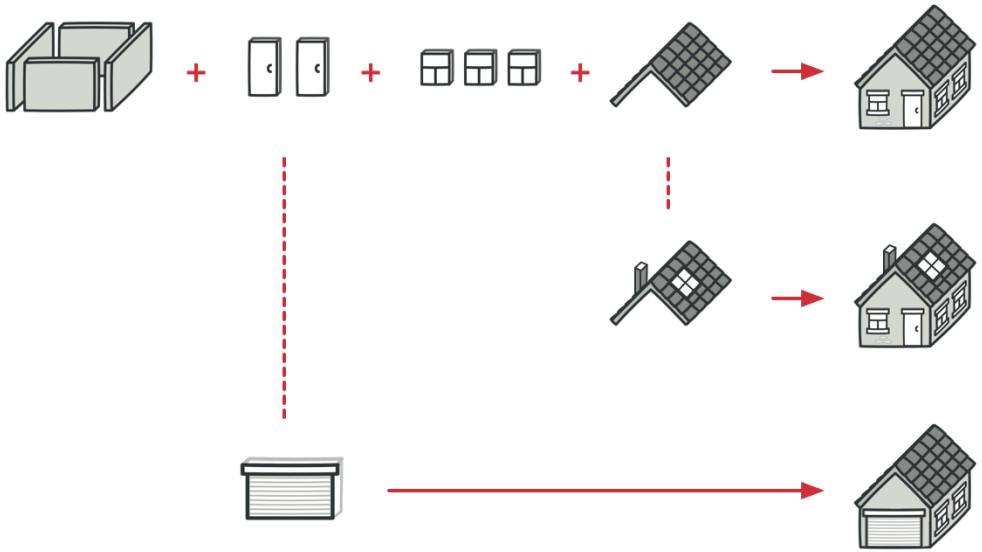
- "To handle this problem, let's use Composite Strategy"
- Design reasoning based on:
  - Protected Variation
  - Polymorphism
  - High Cohesion
  - Low Coupling
  - Strategy
  - Composite
  - Factory
  - Singleton

Problem: How to modify a generic design to satisfy specific algorithm

- **Problem:**
  - How to build generic components that are easy to extend and use without replicating code

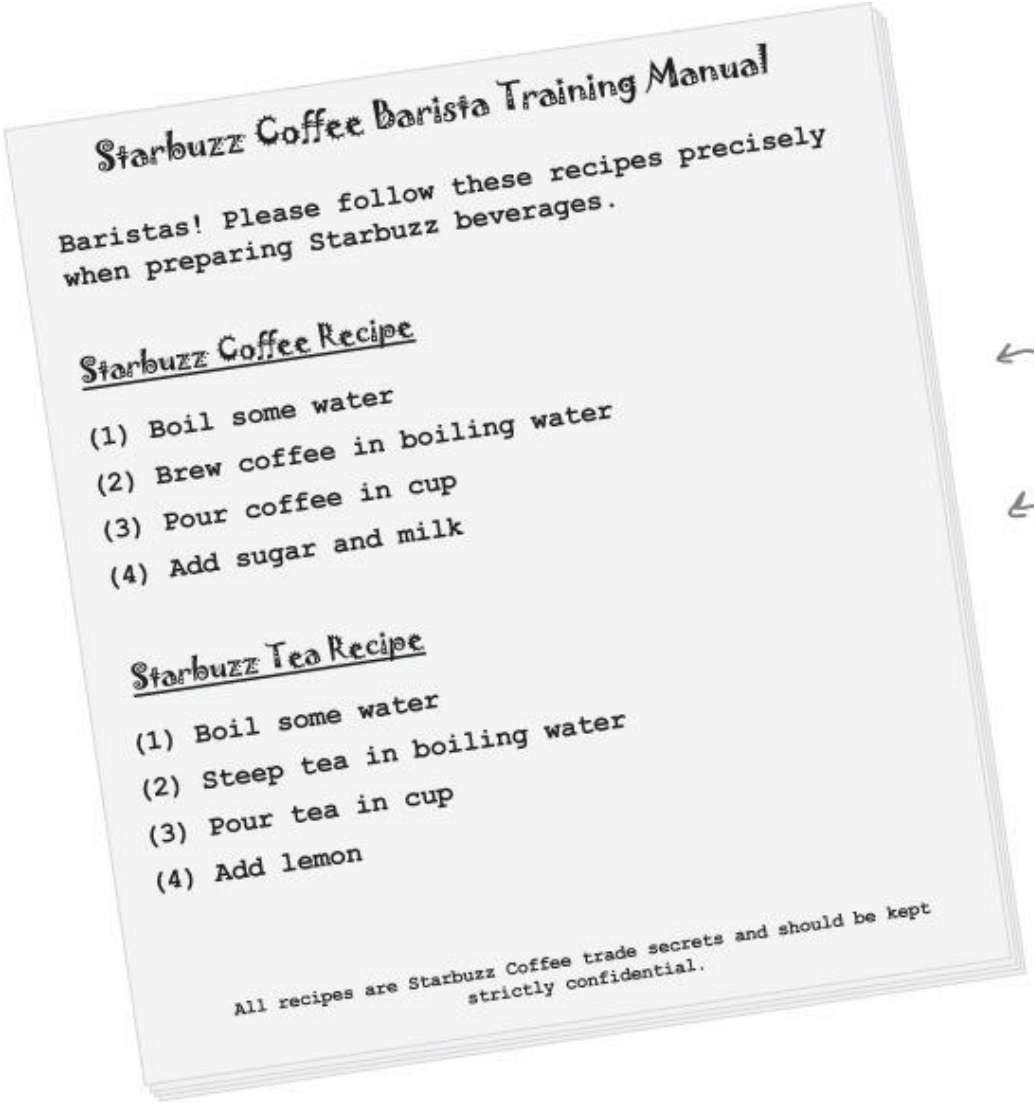
- **Solution (Advice):**
  - Define a family of algorithms, encapsulate each one, and make them interchangeable.
  - Allows the implementation of invariant parts of an algorithm once and leave it to the subclass to implement the behavior that can vary.

Problem: Template (GoF) Revisited from SWEN20003



A typical architectural plan can be slightly altered to better fit the client's needs

Example: Making Tea & Coffee from Caffeine



Starbuzz Coffee Barista Training Manual

Baristas! Please follow these recipes precisely when preparing Starbuzz beverages.

Starbuzz Coffee Recipe

(1) Boil some water
(2) Brew coffee in boiling water
(3) Pour coffee in cup
(4) Add sugar and milk

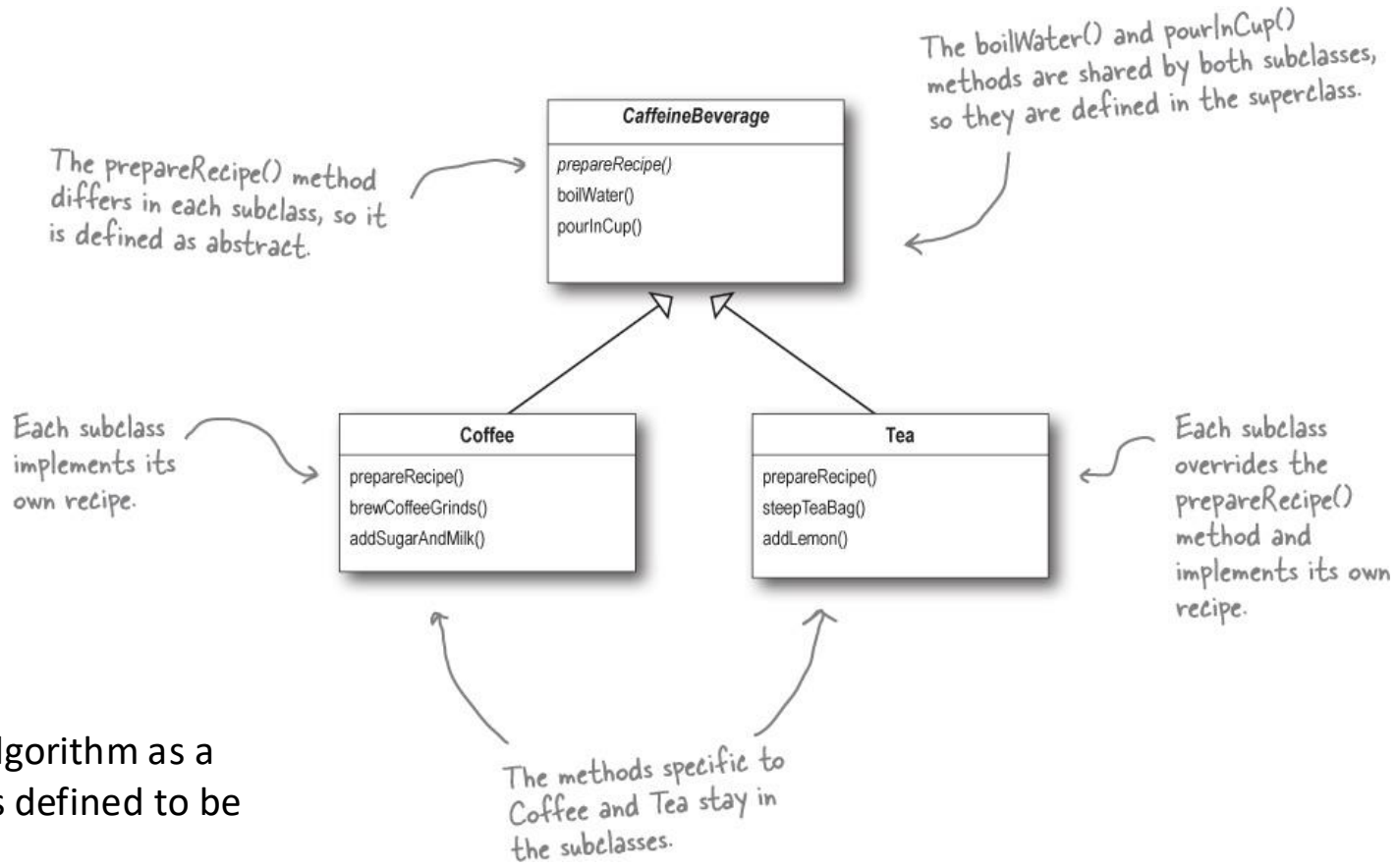
Starbuzz Tea Recipe

(1) Boil some water
(2) Steep tea in boiling water
(3) Pour tea in cup
(4) Add lemon

All recipes are Starbuzz Coffee trade secrets and should be kept strictly confidential.

The recipe for coffee looks a lot like the recipe for tea, doesn't it?

Example: Class diagram



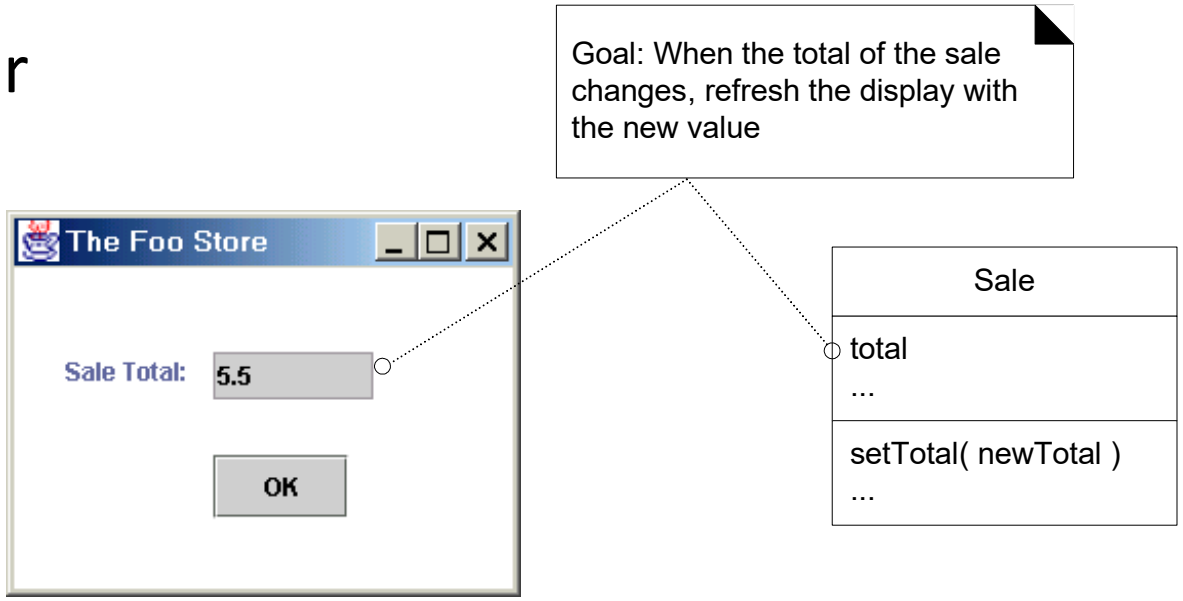
A **template** is a method that defines an algorithm as a set of steps. One or more of these steps is defined to be abstract and implemented by a subclass.

This ensures the algorithm's structure stays unchanged, while subclasses provide some part of the implementation.

Problem 8: Refreshing Sales Total UI

- a GUI window refreshes its display of the sale total when the total changes
- *Sale* object **should not** send a message to a window, asking it to refresh its display
  - Low coupling from other layers to the UI layer

"To handle this problem, let's have an object observing the Sale object"



Observer (aka Publish-Subscribe) (GoF) – Revisited from SWEN20003

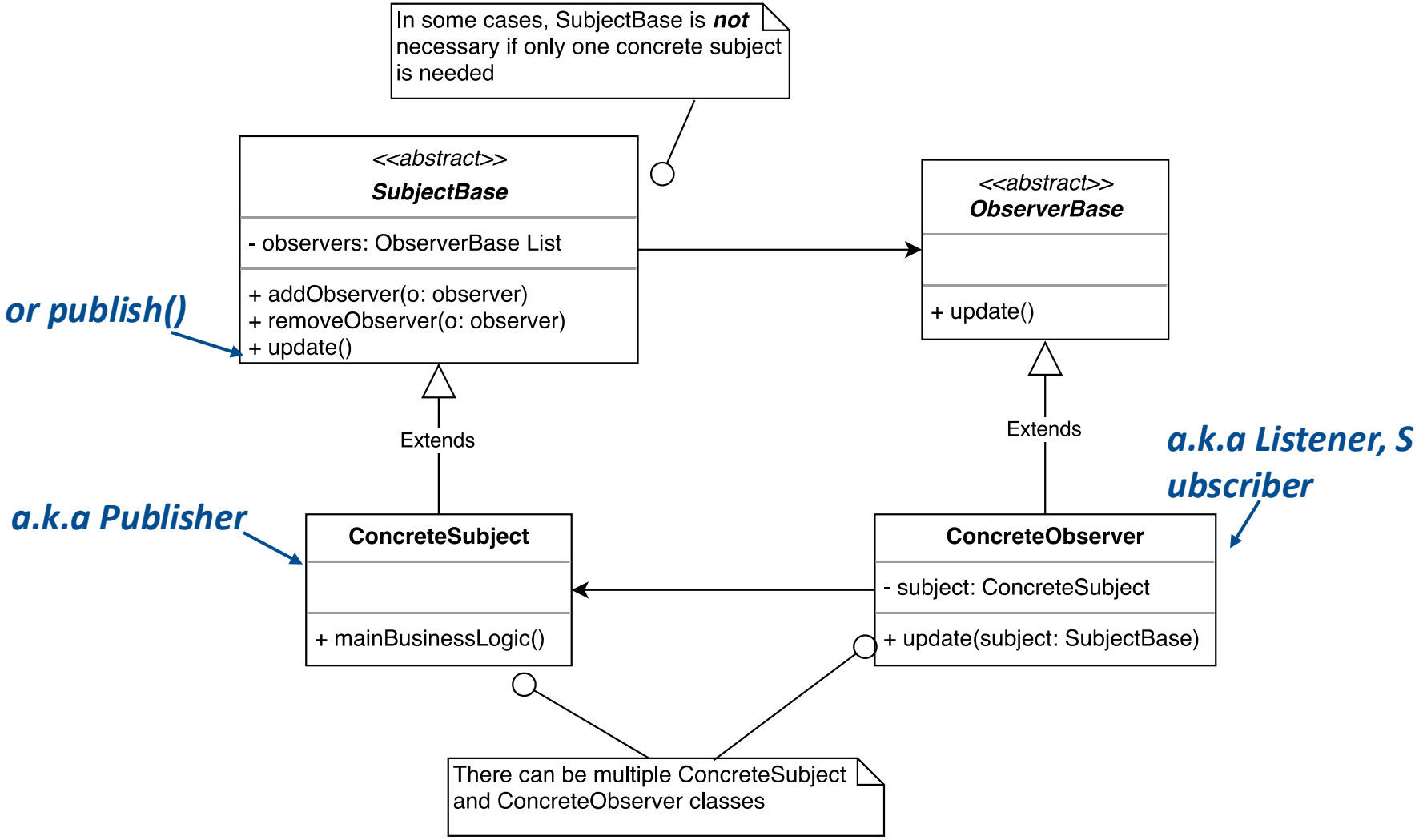
Problem:

- Different kinds of subscriber objects are interested in the state changes or events of a publisher object, and want to react in their own unique way when the publisher generates an event. Moreover, the publisher wants to maintain low coupling to the subscribers. What to do?

Solution (advice):

- Define a “subscriber” or “listener” interface. Subscribers implement this interface. The publisher can dynamically register subscribers who are interested in an event and notify them when an event occurs.

Observer: Generalised Structure



Summary of related GoF Patterns

- **Adapter** provides a different interface to its subject. **Decorator** provides an enhanced interface.
- **Composite** and **Decorator** have similar structure, reflecting the fact that both rely on recursive composition to organize an open-ended number of objects.
- **Composite**, unlike **Decorator**, is not focused on embellishment but on representation. These intents are distinct but complementary. Consequently, Composite and Decorator are often used in concert.
- **Decorator** lets you change the skin of an object. **Strategy** lets you change the guts.

Summary (Contd.)

- **Template Method** and **Strategy** are two design patterns that solve the problem of separating a generic algorithm from a detailed design.
- **Template Method** pattern uses Inheritance. **Strategy** pattern uses Delegation.
- Most of the times, subclasses calls methods from super class but in template pattern, superclass template method routinely calls methods from subclasses; this is known as the **Hollywood Principle** - “don’t call us, we’ll call you.”
- Calling a **Strategy** does not necessarily involve a structured process as in **Template Method**.

THE UNIVERSITY OF MELBOURNE

Lecture Identification

Coordinator: Dr. Philip Dart

Lecturer: Dr Patanamon Thongtanunam

Semester: S1 2025

© University of Melbourne 2025

These slides include materials from:

Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, Third Edition, by Craig Larman, Pearson Education Inc., 2005.